

Tooth Sequence Generation: Temperley Tooth Enumeration

Summary of `generate_tooth_sequences.py`

(a) Theory

The tooth generating functions

For each base length r , the paper defines a generating function

$$h_r(q) = \sum_T q^{|T|},$$

where the sum runs over all valid teeth T of base r , and $|T|$ is the total number of cubes in the tooth. The coefficient

$$T(r, N) = [q^N]h_r(q)$$

therefore counts the number of teeth of base length r and total cube count N .

The two-term recurrence

The script is based on the recurrence

$$(1 - q^r)h_r(q) = 2qh_{r-1}(q) - q^2h_{r-2}(q), \quad r \geq 2,$$

with initial conditions

$$h_0(q) = 1, \quad h_1(q) = \frac{q}{1 - q} = q + q^2 + q^3 + \dots$$

This recurrence is the computational form of the paper's tooth-enumeration result. It expresses h_r in terms of the two previous base lengths, so all rows of the coefficient triangle can be built successively from $r = 0$ and $r = 1$.

Coefficient recurrence

Writing

$$h_r(q) = \sum_{N \geq 0} T(r, N)q^N,$$

the recurrence may be used coefficient-by-coefficient. If

$$R_r(q) = 2qh_{r-1}(q) - q^2h_{r-2}(q),$$

then

$$(1 - q^r)h_r(q) = R_r(q),$$

so the coefficient of q^N satisfies

$$T(r, N) = [q^N]R_r(q) + T(r, N - r),$$

where $T(r, N - r) = 0$ if $N < r$. In expanded form this is

$$T(r, N) = 2T(r - 1, N - 1) - T(r - 2, N - 2) + T(r, N - r).$$

This is exactly the recurrence implemented in the script.

Why exact rational arithmetic is used

Although the coefficients in this application are integers, the script stores coefficients as `Fraction` objects. This ensures that any future extension involving rational generating functions or intermediate rational coefficients remains exact. In the current script, the final values are converted to integers for printing and comparison with the table in the paper.

(b) What the script does

The script computes the coefficient triangle

$$T(r, N) = [q^N]h_r(q)$$

for tooth generating functions up to a specified maximum base length and maximum cube count.

It performs three main tasks:

1. **Coefficient generation:** computes the coefficient lists for $h_r(q)$ for $1 \leq r \leq 7$ and $0 \leq N \leq 30$.
2. **Triangle printing:** prints the rectangular table $T(r, N)$ for $1 \leq r \leq 7$ and $1 \leq N \leq 30$.
3. **Verification against the paper:** checks the computed values against Table 2 of the submitted paper for $1 \leq r \leq 7$ and $1 \leq N \leq 15$.

The script therefore acts as a compact reproducibility check for the numerical coefficient table in the manuscript, while also extending that table from $N = 15$ to $N = 30$.

(c) How the script does it

Initial conditions

The dictionary `h` stores one coefficient list for each base length r .

The script initializes

```
h[0] = [1, 0, 0, ..., 0]
h[1] = [0, 1, 1, ..., 1]
```

corresponding to

$$h_0(q) = 1,$$

and

$$h_1(q) = q + q^2 + q^3 + \dots$$

Thus $h[r][N]$ represents the coefficient $[q^N]h_r(q)$.

Recursive construction of each row

For each $r = 2, 3, \dots, \text{max_r}$, the script constructs a new coefficient list `coeffs` of length `max_N + 1`.

For each coefficient index N , it first forms the right-hand side coefficient

$$[q^N]\{2qh_{r-1}(q) - q^2h_{r-2}(q)\}.$$

In code, this is

```
rhs_n = 0
if n >= 1:
    rhs_n += 2 * h[r-1][n-1]
if n >= 2:
    rhs_n -= h[r-2][n-2]
```

This corresponds to the shifts induced by multiplication by q and q^2 .

Division by $1 - q^r$

The recurrence contains the factor $(1 - q^r)$ on the left. Instead of performing symbolic division, the script solves coefficient-by-coefficient:

$$T(r, N) = [q^N]R_r(q) + T(r, N - r).$$

In code:

```
if n >= r:
    coeffs[n] = rhs_n + coeffs[n-r]
else:
    coeffs[n] = rhs_n
```

This is equivalent to multiplying the right-hand side by

$$\frac{1}{1 - q^r} = 1 + q^r + q^{2r} + q^{3r} + \dots,$$

but avoids constructing a symbolic power series explicitly.

Output formatting

The function `print_sequences(h, max_r, max_N)` prints two views of the data:

1. a rectangular table with rows indexed by r and columns indexed by N ;
2. individual sequences for each fixed r , starting at $N = r$, since no tooth of base r can have fewer than r cubes.

For example, the row $r = 1$ begins

$$1, 1, 1, 1, \dots,$$

because there is exactly one single-column tooth of size N for every $N \geq 1$.

Table 2 verification

The script contains a hard-coded copy of the manuscript's Table 2 for

$$1 \leq r \leq 7, \quad 1 \leq N \leq 15.$$

It compares every computed value with the corresponding table entry. If any discrepancy is found, it prints a message of the form

```
MISMATCH at r=..., N=...: computed ..., expected ...
```

If all entries agree, it prints

```
All values match Table 2.
```

(d) Output produced

Coefficient triangle

With the default settings

```
max_r = 7
max_N = 30
```

the script prints the triangle

$$T(r, N) = [q^N] h_r(q)$$

for $1 \leq r \leq 7$ and $1 \leq N \leq 30$.

The first part of this output reproduces the paper's Table 2. For $N = 1$ to 15, the rows are:

$r \backslash N$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

$r \backslash N$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	0	0	1	3	5	8	12	16	21	27	33	40	48	56	65
4	0	0	0	1	4	7	12	20	30	42	58	77	100	127	158
5	0	0	0	0	1	5	9	16	28	45	68	98	137	188	251
6	0	0	0	0	0	1	6	11	20	36	60	95	144	208	296
7	0	0	0	0	0	0	1	7	13	24	44	75	122	191	286

The script then continues the same rows to $N = 30$, producing additional terms not displayed in the manuscript table.

Individual fixed- r sequences

The script also prints each fixed-base sequence beginning at its first non-zero entry:

```
r=1: [1, 1, 1, ...]
r=2: [1, 2, 3, ...]
r=3: [1, 3, 5, 8, ...]
...
```

These are the rows $T(r, N)$ with the initial forced zeros omitted.

Verification message

Finally, the script verifies the printed values against the paper's Table 2. For the submitted version of the script, the expected outcome is

```
All values match Table 2.
```

This confirms that the coefficient recurrence has been implemented consistently with the values reported in the manuscript.

(e) What we learn

The recurrence is sufficient to reproduce the table. The script confirms that the two-term recurrence for $h_r(q)$ generates all entries in the manuscript's coefficient triangle for $1 \leq r \leq 7$ and $1 \leq N \leq 15$.

The implementation is coefficient-based, not symbolic. The script does not manipulate rational functions directly. Instead, it propagates finite coefficient lists up to $N = 30$. This is simpler, faster, and less error-prone for table generation.

The factor $(1 - q^r)^{-1}$ is handled by a same-row recurrence. The update

$$T(r, N) = 2T(r - 1, N - 1) - T(r - 2, N - 2) + T(r, N - r)$$

is the computational core of the script. It implements the geometric-series expansion of $(1 - q^r)^{-1}$ without explicitly expanding the denominator.

The table has a clear combinatorial interpretation. Each entry $T(r, N)$ counts teeth with fixed base r and fixed total cube count N . Reading across rows gives fixed-base sequences; reading down columns gives the distribution of all teeth of fixed size N by base length.

The script is primarily a reproducibility and extension tool. Its most important role is to verify Table 2 of the submitted paper and extend the same computation beyond the displayed range. It is not a Monte Carlo script, nor an asymptotic analysis script; it is an exact finite coefficient-generation script.